Exp No: 6

ERROR DETECTION AND CORRECTION USING HAMMING CODE

Aim

To write a program to implement error and correction using Hamming code

ALGORITHM

1. Start the program
2. Read the number of 4-bit data block (n) forming the input data stream.
3. For each 4-bit block D = d_1 d_2 d_3 , perform Hamming code
   a. Arrange 4 bit with parity bit at position 1, 2 & 4 and data bit at position 3, 5, 6 & 7.
   b. Compute the parity bit using even parity:

$P_1 = d_1$ XOR d_2 XOR d_4

$P_2 = d_1$ XOR d_3 XOR d_4

$P_3 = d_2$ XOR d_3 XOR d_4

4. Simulate transmission by introducing a single-bit error at a chosen bit position in the code word.
5. At the receiver, position = C_4 C_2 C_1 (binary numbers formed from the check bit)
7. If Error position = 0, no error has occured. Else, the bit at "Error Position" is inverted (complemented) to connect the error.
8. Extract the original 4 data bits from the corrected 7-bit code code.
9. Repeat step 3 to 8 for all n blocks in the data stream.

10 Stop the program.

PROGRAM:

```
def encode(d):
h= [0] * 7
h[2] = d[0]
h[4] = d[1]
h[5] = d[2]
h[6] = d[3]

h[0] = h[2] ^ h[4] ^ h[6]
h[1] = h[2] ^ h[5] ^ h[6]
h[3] = h[4] ^ h[5] ^ h[6]

return h

def detect_and_correct(h):
    c1 = h[0] ^ h[2] ^ h[4] ^ h[6]
    c2 = h[1] ^ h[2] ^ h[5] ^ h[6]
    c4 = h[3] ^ h[4] ^ h[5] ^ h[6]

    error_pos = c1 * 1 + c2 *2 + c4 * 4

    if error_pos !=0:
        print(f" Error detected at bit position:"
              f" {error_pos} → correcting..."

    )
        h[error_pos-1] ^ =1

    else:
        print (" NO error detected.")

    return error_pos
```

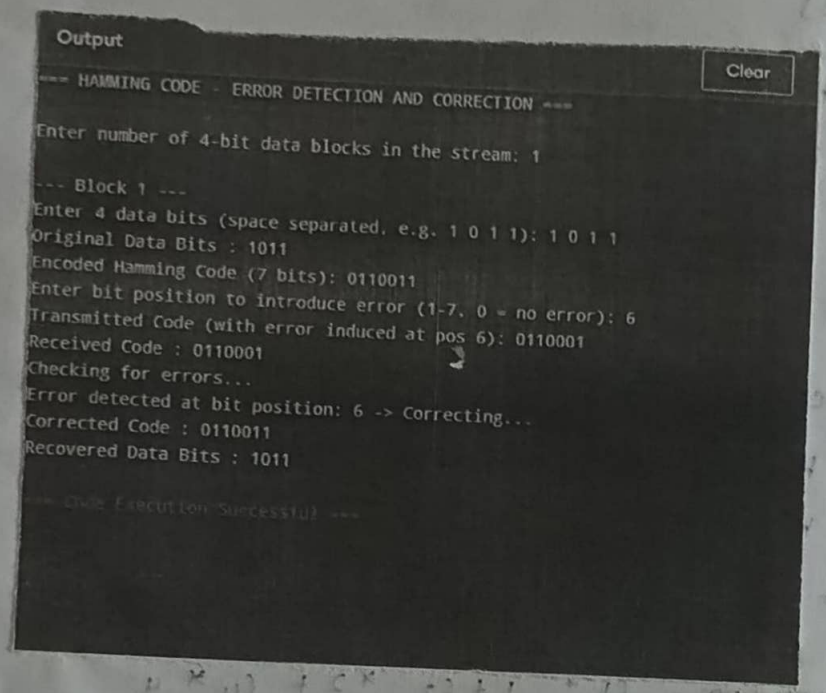
```
def to_str(bits):
    return "".join(str(b) for b in bits)

def main():
    print("=== HAMMING CODE - ERROR DETECTION AND
                        CORRECTION ===\n")
    n = int(input("Enter number of 4-bit data blocks
    in the stream: "))

    for b in range(1, n+1):
        print(f"\n-- Block {b} ---")
        d = list(
            map(int, input("Enter. split()
        )
    )
    h = encode(d)
    print("Orginal Data Bits: {to_str(d)}")
    pos = int(input())
    if 1 <= pos <= 7:
        h[pos-1] ^= 1
    else:
        print("Transmitted code (no error induced): {to_str(h)}")
        print("Checking for error...")
        detect_and_correct(h)
        print(f"Correct code: {to_str(h)}")
        rd = [h[2], h[4], h[5], h[6]]
        print(f"Recovered Data Bits: {to_str(rd)}")

if __name__ == "__main__":
    main()
```

Result: The program to implement error detection & correction using the Hamming code concept was successfully written in python, executed & verified using a sample input data stream.


Output
Clear
=== HAMMING CODE - ERROR DETECTION AND CORRECTION ===
Enter number of 4-bit data blocks in the stream: 1
--- Block 1 ---
Enter 4 data bits (space separated, e.g. 1 0 1 1): 1 0 1 1
Original Data Bits : 1011
Encoded Hamming Code (7 bits): 0110011
Enter bit position to introduce error (1-7, 0 = no error): 6
Transmitted Code (with error induced at pos 6): 0110001
Received Code : 0110001
Checking for errors...
Error detected at bit position: 6 -> Correcting...
Corrected Code : 0110011
Recovered Data Bits : 1011
=== Code Execution Successful ===